LogInsight AI

Intelligent Log Analyzer

Project Description:

1. Project Overview :

LogInsight AI is an intelligent log analysis platform designed for Enterprise IT operations. The system allows engineers and IT teams to upload system log files and automatically receive a plain-language summary of detected issues, a severity-ranked list of errors, and step-by-step remediation suggestions - all generated by an AI agent that combines deterministic log parsing tools with a generative LLM.

The core innovation of LogInsight AI is its Tool-Augmented LLM architecture: deterministic Python tools handle structured parsing and error detection with 100% repeatability, while the LLM (Groq's Llama 3.3) is used exclusively for explanation and remediation generation. This division ensures reliability without sacrificing the human-readable quality of the output.

All analysis results are stored in a Supabase cloud database, providing persistent job history that team members can retrieve at any time. A clean web-based frontend deployed on Vercel provides drag-and-drop log upload, dark mode, and a history browser.

2. Problem Statement :

Enterprise IT systems generate thousands of log lines per minute across servers, applications, databases, and network devices. When an incident occurs, engineers are required to manually search through these large files to identify error patterns, understand root causes, and determine fixes. This process is slow, inconsistent, and does not scale.

Specific challenges include:

- Engineers spend hours manually grepping through log files during critical incidents, increasing Mean Time to Resolution (MTTR).
- Error interpretation relies on tribal knowledge - senior engineers understand log patterns that newer team members do not.
- There is no systematic classification of errors, leading to inconsistent triage across teams.
- Logs from different systems use different formats (syslog, JSON, Apache), making unified analysis difficult.
- Past incidents and their fixes are not recorded in a searchable, structured way.

LogInsight AI addresses all of these problems by automating the triage pipeline: from raw log upload to structured error detection to AI-generated explanation and fix suggestions, with full history stored for future reference.

3. OBJECTIVES :

TECHNICAL OBJECTIVES

- Build a REST API using FastAPI that accepts log file uploads and returns structured analysis results.
- Implement deterministic log parsing supporting four formats: Generic, JSON, Apache/Nginx, and Syslog RFC 5424.
- Build a rule-based error detection engine covering 10 common error categories with severity ranking.
- Expose both tools via the Model Context Protocol (MCP) using FastMCP, making them callable by an LLM agent.
- Integrate Groq's Llama 3.3 model to generate plain-language summaries and step-by-step fix suggestions.
- Store all analysis jobs in Supabase PostgreSQL for persistent history and retrieval.

PERFORMANCE OBJECTIVES

- Analyze a standard log file (up to 10MB) in under 30 seconds end-to-end.
- Return accurate error detection results with zero false negatives for the 10 defined patterns.
- Ensure PII is scrubbed from all log content before it is sent to any external API.

LEARNING OUTCOMES

- Understand the Tool-Augmented LLM architecture and why it is superior to sending raw data to an LLM.
- Gain hands-on experience with FastAPI, MCP (Model Context Protocol), and FastMCP.
- Learn how to integrate cloud services (Groq, Supabase, Vercel) into a production-grade Python application.
- Practice writing deterministic Python tools that are tested with pytest.

4. CORE TECHNOLOGIES USED

Technology	Purpose
Python 3.11+	Primary backend language for all server-side logic
FastAPI	Web framework for REST API endpoints with auto-generated Swagger docs
FastMCP	MCP server framework - exposes parse_logs() and detect_errors() as LLM-callable tools
Groq API (Llama 3.3)	Free-tier LLM inference for generating summaries and suggested fixes
Supabase	Managed PostgreSQL cloud database for storing job history and results
HTML / CSS / JavaScript	Vanilla frontend for drag-drop upload, results display, and history view
Vercel	Free cloud hosting platform for the frontend web application
pyparsing + drain3	Log parsing libraries for tokenizing and auto-detecting log formats
pandas	Data manipulation for the error detection rule engine
scrubadub	PII scrubbing - removes emails, IPs, passwords before LLM call
pytest + httpx	Testing framework for unit and integration tests

5. KEY FEATURES

MULTI-FORMAT LOG PARSING

- Automatically detects log format from the first line of the file - no manual configuration needed.
- Supports four formats: Generic timestamped logs, JSON structured logs, Apache/Nginx access logs, and Syslog RFC 5424.
- Each parsed line is converted into a structured LogEvent object with timestamp, level, source, and message fields.

RULE-BASED ERROR DETECTION

- Scans all parsed log events against 10 predefined regex patterns covering the most common enterprise error categories.
- Detected issues are ranked by severity (CRITICAL → ERROR → WARNING) and then by frequency (count).
- Each issue includes first seen, last seen timestamps and up to 3 sample messages for context.

AI-GENERATED SUMMARIES AND FIXES

- Detected issues and a sample of 30 log events are sent to Groq's Llama 3.3 model.
- The LLM generates a 2–5 sentence plain-English incident summary and a step-by-step fix for each detected issue.
- Strict JSON output format ensures the response is always machine-parseable.

MCP TOOL INTEGRATION (FASTMCP)

- Both parse_logs() and detect_errors() are exposed as MCP tools via a FastMCP server.
- The LLM agent can call these tools in a structured, auditable way through the Model Context Protocol.
- Every tool call is logged with inputs and outputs, supporting full traceability.

PERSISTENT JOB HISTORY (SUPABASE)

- Every analysis job is saved to a Supabase PostgreSQL jobs table with status, results, and timestamp.
- Users can retrieve past analyses by job ID or browse recent history via the /history endpoint.
- Enables team-wide access to historical incident analysis records.

WEB FRONTEND

- Drag-and-drop log file upload with client-side validation.
- Dark mode toggle, responsive design, and animated results rendering.
- Job history browser with click-to-expand past results and JSON export.

6. TARGET USERS

- Site Reliability Engineers (SREs) - investigating production incidents and need fast root-cause identification.
- DevOps Engineers - monitoring CI/CD pipeline logs and deployment failure analysis.
- IT Operations Analysts - performing routine infrastructure health reviews across multiple systems.
- Security Analysts - scanning authentication and access logs for anomalous patterns and breach indicators.
- Junior engineers and on-call staff - who may lack the expertise to interpret raw log errors without AI-assisted explanation.

7. APPLICATION SCENARIOS / USE CASES

SCENARIO 1: PRODUCTION DATABASE OUTAGE

An SRE is paged at 2 AM for a service outage. They upload the application log from the past hour. LogInsight AI detects repeated Connection Refused errors to the database server, an OOM event 5 minutes before the outage, and 12 HTTP 500 errors. The AI summary explains the memory exhaustion caused the DB connection pool to crash, and the suggested fix walks through increasing heap size and restarting the connection pool with backoff.

SCENARIO 2: AUTHENTICATION ATTACK DETECTION

A security analyst uploads an Nginx access log from the past 24 hours. The system detects 847 AUTH FAILURE events from the same IP address within a 10-minute window. The AI summary identifies this as a brute-force attack pattern and the suggested fix recommends IP blocking, rate limiting, and enabling fail2ban.

SCENARIO 3: DEPLOYMENT FAILURE ANALYSIS

A DevOps engineer runs a deployment and sees the service fail to start. They upload the container startup log. LogInsight AI detects a Missing File error (configuration file not found), a Stacktrace from the app initialization, and a Connection Failure to a downstream service. The AI summary explains the config was not mounted in the container and the fix is a step-by-step Docker volume configuration correction.

SCENARIO 4: DISK CAPACITY PLANNING

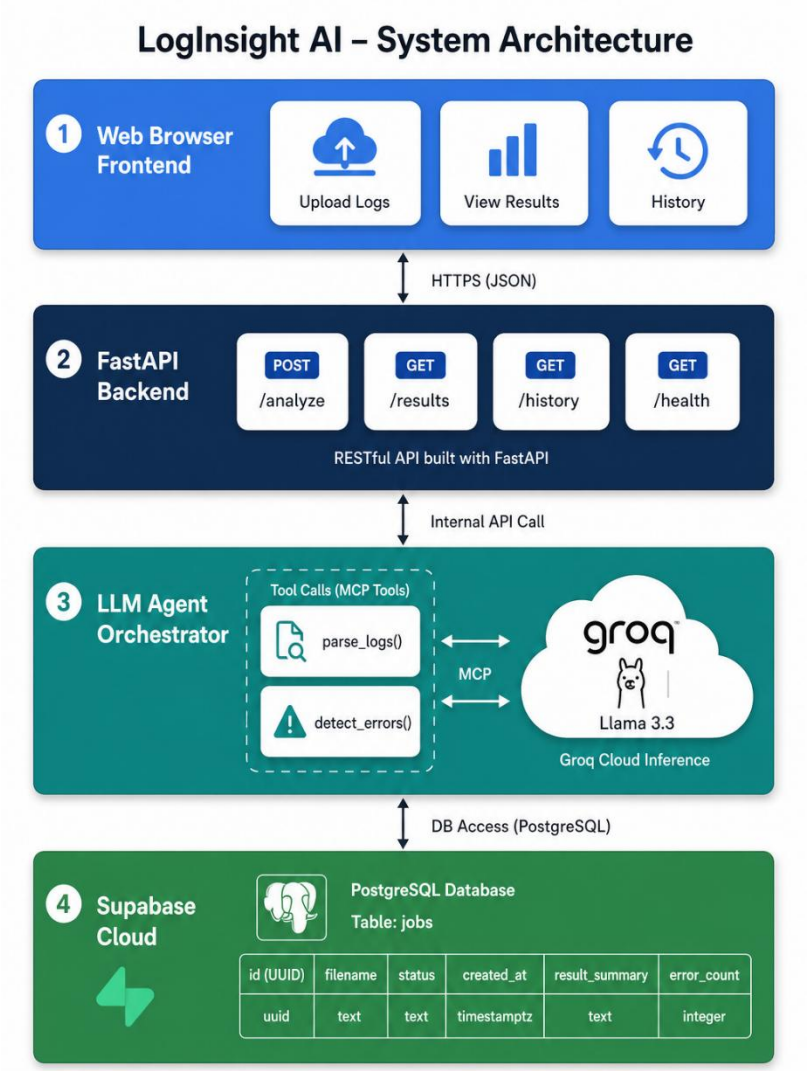
An IT operations analyst uploads a week of cron job logs. The system detects recurring Disk Full warnings that appear every Tuesday and Friday. The AI summary identifies a log rotation misconfiguration as the root cause and provides a step-by-step fix for setting up automated log cleanup.

8. Technical Architecture

SYSTEM ARCHITECTURE OVERVIEW

LogInsight AI follows a three-tier architecture: a frontend layer (browser-based UI), a backend layer (FastAPI REST API), and a data/AI layer (Supabase + Groq). The distinguishing feature of the architecture is the Tool-Augmented LLM Agent pattern embedded in the backend.

ARCHITECTURE DIAGRAM



FRONTEND LAYER

The frontend is a single-page web application built with HTML, CSS, and Vanilla JavaScript. It provides a drag-and-drop interface for log file uploads, renders analysis results with severity-color-coded issue cards, and includes a history browser for past analyses. Dark mode and responsive design are implemented using CSS custom properties.

BACKEND LAYER (FASTAPI)

The FastAPI application serves four REST endpoints. The primary endpoint POST /analyze receives an uploaded log file, saves a job record to Supabase, calls the agent orchestrator, and returns the full analysis result. GET /results/{job_id} retrieves a past result. GET /history returns the 20 most recent jobs. GET /health returns a service status check.

AGENT ORCHESTRATOR

The orchestrator is the central processing unit. It calls parse_logs() to convert the raw log file into structured events, then calls detect_errors() on those events to produce a ranked issue list. It then constructs a structured prompt with the issue data and 30 sample log events, scrubs PII, and calls the Groq API. The LLM response is parsed from JSON and returned.

MCP LAYER (FASTMCP)

Both parse_logs() and detect_errors() are wrapped with the @mcp.tool() decorator in a FastMCP server. This exposes them over the Model Context Protocol - a standardized interface that allows LLMs to discover and call tools in a structured, schema-validated way. The MCP server runs as a separate process on the same machine.

DATABASE LAYER (SUPABASE)

Supabase provides a managed PostgreSQL instance. A single table - jobs - stores every analysis with columns for job ID (UUID), filename, status (processing/complete/failed), summary (text), issues (JSONB array), fixes (JSONB array), error message, and created_at timestamp.

9. PRE-REQUISITES

SOFTWARE REQUIREMENTS

- Python 3.11 or higher - the primary runtime for all backend code.
- pip - Python package manager for installing dependencies.
- Git - for cloning the repository.

- VS Code - recommended IDE with Python extension for development on Windows.
- A modern web browser (Chrome, Edge, Firefox) for accessing the frontend.
- A Groq account (free) at console.groq.com to obtain an API key.
- A Supabase account (free) at supabase.com to create a project and obtain URL/key.

LIBRARIES AND DEPENDENCIES

Install all dependencies using: pip install -r requirements.txt

Library	Purpose
fastapi	Web framework for REST API
uvicorn	ASGI server to run FastAPI on Windows
fastmcp	MCP server framework for exposing tools
groq	Official Groq Python client for LLM inference
supabase	Official Supabase Python client for DB operations
pyparsing	Log line tokenization and parsing
drain3	Auto-detection of unknown log formats
pandas	Data manipulation in the error detection engine
scrubadub	PII scrubbing before LLM calls
python-dotenv	Load .env secrets into environment variables
pytest + httpx	Unit testing and async API test client

HARDWARE REQUIREMENTS

- CPU: Intel Core i5 / AMD Ryzen 5 or better (2 GHz+).
- RAM: Minimum 8 GB recommended.
- Storage: 500 MB free disk space for project and dependencies.
- Network: Stable internet connection required for Groq API and Supabase calls.
- GPU: Not required - all inference is handled by Groq cloud.

10. PRIOR KNOWLEDGE REQUIRED

PROGRAMMING LANGUAGE

- Basic Python - variables, functions, classes, and modules.
- Familiarity with JSON data format and how to parse/serialize it in Python.
- Basic understanding of error handling (try/except) and file I/O.

WEB & API CONCEPTS

- Understanding of HTTP methods - GET (fetch data), POST (send data).
- Basic knowledge of REST APIs - what endpoints are and how request/response works.
- Familiarity with how web forms and file uploads work in HTML.

DATABASE FUNDAMENTALS

- Basic knowledge of relational databases - tables, rows, columns.
- Understanding of CRUD operations - Create, Read, Update, Delete.
- Familiarity with cloud databases (no local server installation needed for Supabase).

AI / LLM CONCEPTS

- Basic understanding of what a Large Language Model (LLM) is and how prompting works.
- Awareness of LLM limitations - hallucination, context window limits.
- Understanding of why deterministic tools are combined with LLMs (reliability + readability).

11. PROJECT WORKFLOW

MILESTONE 1: SYSTEM DESIGN AND SETUP

- Activity 1.1: Define problem statement and identify target users.
- Activity 1.2: Design system architecture and data flow diagram.
- Activity 1.3: Set up Python virtual environment, install dependencies.
- Activity 1.4: Create Supabase project and execute schema SQL.

MILESTONE 2: CORE TOOL DEVELOPMENT

- Activity 2.1: Implement parse_logs() with support for all 4 log formats.
- Activity 2.2: Implement detect_errors() with 10 regex patterns and severity ranking.
- Activity 2.3: Write pytest unit tests for both tools covering edge cases.
- Activity 2.4: Wrap tools in FastMCP server with @mcp.tool() decorators.

MILESTONE 3: AGENT AND API DEVELOPMENT

- Activity 3.1: Implement agent/orchestrator.py - full analysis pipeline.
- Activity 3.2: Add PII scrubbing and Groq LLM integration.
- Activity 3.3: Build FastAPI endpoints - /analyze, /results, /history, /health.
- Activity 3.4: Integrate Supabase job storage - save, complete, fail, get, list.

MILESTONE 4: FRONTEND DEVELOPMENT

- Activity 4.1: Build HTML page with drag-and-drop upload zone.
- Activity 4.2: Implement JavaScript API calls and result rendering.

- Activity 4.3: Add dark mode, responsive design, and history tab.
- Activity 4.4: Deploy frontend to Vercel.

MILESTONE 5: TESTING AND DOCUMENTATION

- Activity 5.1: Run full pytest suite and fix any failures.
- Activity 5.2: Test end-to-end flow with real log files.
- Activity 5.3: Write README.md and prepare final documentation.

MILESTONE 1: SYSTEM DESIGN AND SETUP

ACTIVITY 1.1: DEFINE PROBLEM STATEMENT AND IDENTIFY TARGET USERS

The problem of manual log analysis in enterprise systems was studied, focusing on challenges such as large log volumes, lack of automation, and dependency on expert knowledge. Target users including Site Reliability Engineers (SREs), DevOps engineers, IT operations teams, and security analysts were identified based on real-world use cases.

ACTIVITY 1.2: DESIGN SYSTEM ARCHITECTURE AND DATA FLOW DIAGRAM

A layered system architecture was designed consisting of frontend, backend, LLM agent, and database layers. The Tool-Augmented LLM architecture was selected to separate deterministic processing from AI-based reasoning. A data flow diagram was created to represent the step-by-step flow from log upload to final result generation.

ACTIVITY 1.3: SET UP PYTHON VIRTUAL ENVIRONMENT AND INSTALL DEPENDENCIES

A virtual environment was created using python -m venv venv to isolate project dependencies. Required libraries such as FastAPI, FastMCP, Groq client, Supabase client, pandas, and pytest were installed using pip install -r requirements.txt. Environment variables were configured using a .env file.

ACTIVITY 1.4: CREATE SUPABASE PROJECT AND EXECUTE SCHEMA SQL

A Supabase cloud project was created to manage the PostgreSQL database. The database schema was defined in supabase_schema.sql and executed using the Supabase SQL editor. The jobs table was created to store job metadata, analysis results, and timestamps.

MILESTONE 2: CORE TOOL DEVELOPMENT

ACTIVITY 2.1: IMPLEMENT PARSE_LOGS() WITH MULTI-FORMAT SUPPORT

The parse_logs() function was developed to convert raw log files into structured data. It supports multiple formats including Generic logs, JSON logs, Apache/Nginx logs, and Syslog (RFC 5424). The function detects the format automatically and outputs structured log events with fields like timestamp, level, source, and message.

ACTIVITY 2.2: IMPLEMENT DETECT_ERRORS() WITH REGEX-BASED RULES

The detect_errors() function was implemented using 10 predefined regex patterns to identify common system errors such as connection failures, memory issues, disk errors, and authentication failures. Each detected issue is assigned a severity level (CRITICAL, ERROR, WARNING) and ranked based on severity and frequency.

ACTIVITY 2.3: WRITE PYTEST UNIT TESTS COVERING EDGE CASES

Unit tests were written using pytest to validate both parsing and error detection logic. A total of 14 test cases were implemented covering various log formats, edge cases, and error scenarios to ensure reliability and correctness of the tools.

ACTIVITY 2.4: WRAP TOOLS IN FASTMCP SERVER

The tools were exposed as MCP-compatible functions using the FastMCP framework. The @mcp.tool() decorator was used to register parse_logs() and detect_errors() as callable tools. This enabled structured communication between the LLM agent and the tools through the Model Context Protocol.

MILESTONE 3: AGENT AND API DEVELOPMENT

ACTIVITY 3.1: IMPLEMENT AGENT/ORCHESTRATOR.PY (FULL PIPELINE)

The orchestrator module was developed to manage the complete analysis workflow. It sequentially calls parse_logs() and detect_errors(), processes the outputs, and prepares structured input for the LLM. It acts as the central logic controller of the system.

ACTIVITY 3.2: ADD PII SCRUBBING AND GROQ LLM INTEGRATION

Sensitive data such as email addresses, IP addresses, and credentials were removed using PII scrubbing techniques before sending data to the LLM. The Groq API was integrated to run the Llama 3.3 model, which generates plain-language summaries and suggested fixes based on detected issues.

ACTIVITY 3.3: BUILD FASTAPI ENDPOINTS

A REST API was developed using FastAPI with the following endpoints:

- POST /analyze – Upload and analyze log files
- GET /results/{job_id} – Retrieve analysis results
- GET /history – Fetch recent jobs
- GET /health – Check API status

These endpoints handle file uploads, processing, and response delivery.

ACTIVITY 3.4: INTEGRATE SUPABASE JOB STORAGE

Database operations were implemented using Supabase Python SDK. Functions were created to:

- Save new jobs
- Update job status (processing → completed/failed)
- Retrieve job results
- List historical jobs

This ensures persistent storage and retrieval of analysis data.

MILESTONE 4: FRONTEND DEVELOPMENT

ACTIVITY 4.1: BUILD HTML PAGE WITH DRAG-AND-DROP UPLOAD

A user-friendly frontend interface was created using HTML and CSS. It includes a drag-and-drop upload zone for log files and sections to display results and history.

ACTIVITY 4.2: IMPLEMENT JAVASCRIPT API CALLS AND RENDERING

JavaScript was used to handle API requests to the backend. It sends uploaded files to the /analyze endpoint and dynamically updates the UI with results such as summaries, detected issues, and suggested fixes.

ACTIVITY 4.3: ADD DARK MODE, RESPONSIVE DESIGN, AND HISTORY TAB

UI enhancements such as dark mode, responsive layout for different devices, and a history tab were implemented. The history tab allows users to view and retrieve previous analysis results.

MILESTONE 5: TESTING AND DOCUMENTATION

ACTIVITY 5.1: RUN FULL PYTEST SUITE AND FIX ISSUES

All unit tests were executed using pytest to verify system functionality. Any failures or inconsistencies were identified and resolved to ensure robustness of the tools and API.

ACTIVITY 5.2: TEST END-TO-END WORKFLOW WITH REAL LOGS

The complete system was tested using real-world log files. The workflow from file upload → parsing → error detection → LLM summary → database storage was validated to ensure correct integration of all components.

ACTIVITY 5.3: PREPARE DOCUMENTATION AND README

Comprehensive documentation was created including project overview, architecture, setup instructions, API reference, and workflow explanation. A README file was prepared for GitHub to guide users in running and understanding the project.

12. Project Structure

File / Folder	Description
api/main.py	FastAPI application - all 4 REST endpoints
agent/orchestrator.py	LLM agent - calls tools, scrubs PII, calls Groq, returns results
tools/mcp_server.py	FastMCP server - exposes tools to LLM via MCP protocol
tools/parse_logs.py	parse_logs() - converts raw log text to structured LogEvent list
tools/detect_errors.py	detect_errors() - scans events for 10 error patterns, ranks by severity
storage/supabase_client.py	Supabase helper functions - save/complete/fail/get/list jobs
frontend/	HTML, CSS, JS files for the web UI deployed on Vercel
tests/test_tools.py	pytest unit tests - 14 test cases for both tools
supabase_schema.sql	SQL to create the jobs table - run once in Supabase SQL Editor
.env.example	Template showing required environment variables
requirements.txt	All Python dependencies with pinned versions

13. RESULTS

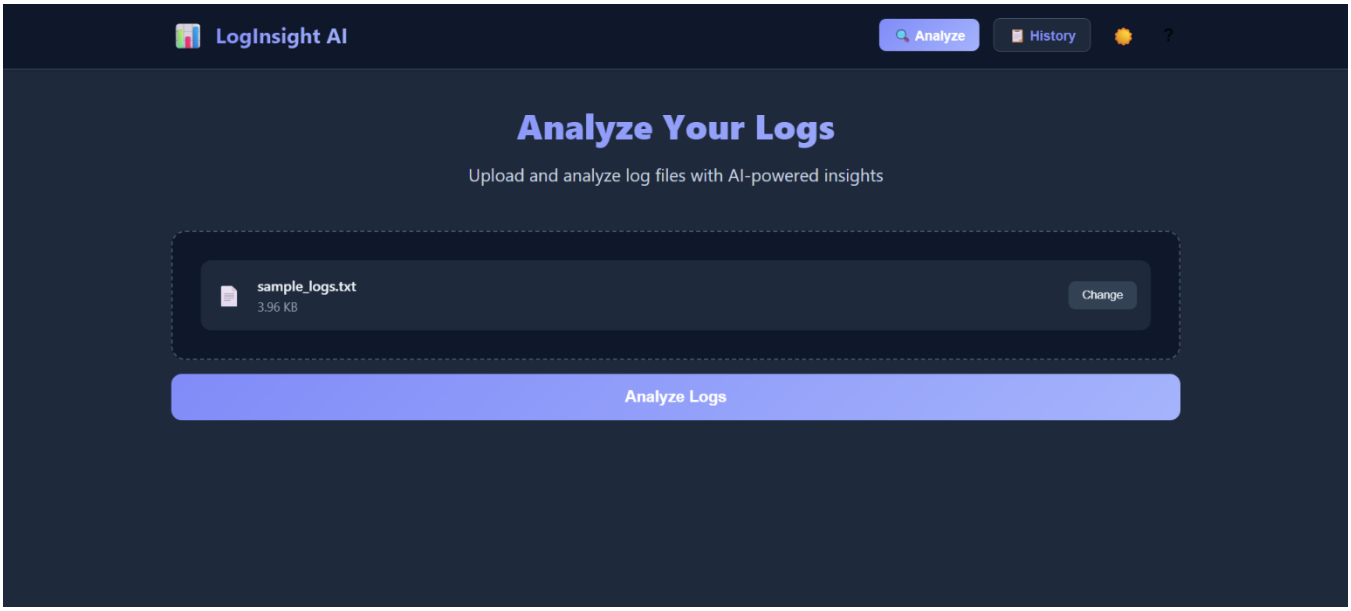
SYSTEM OUTPUT

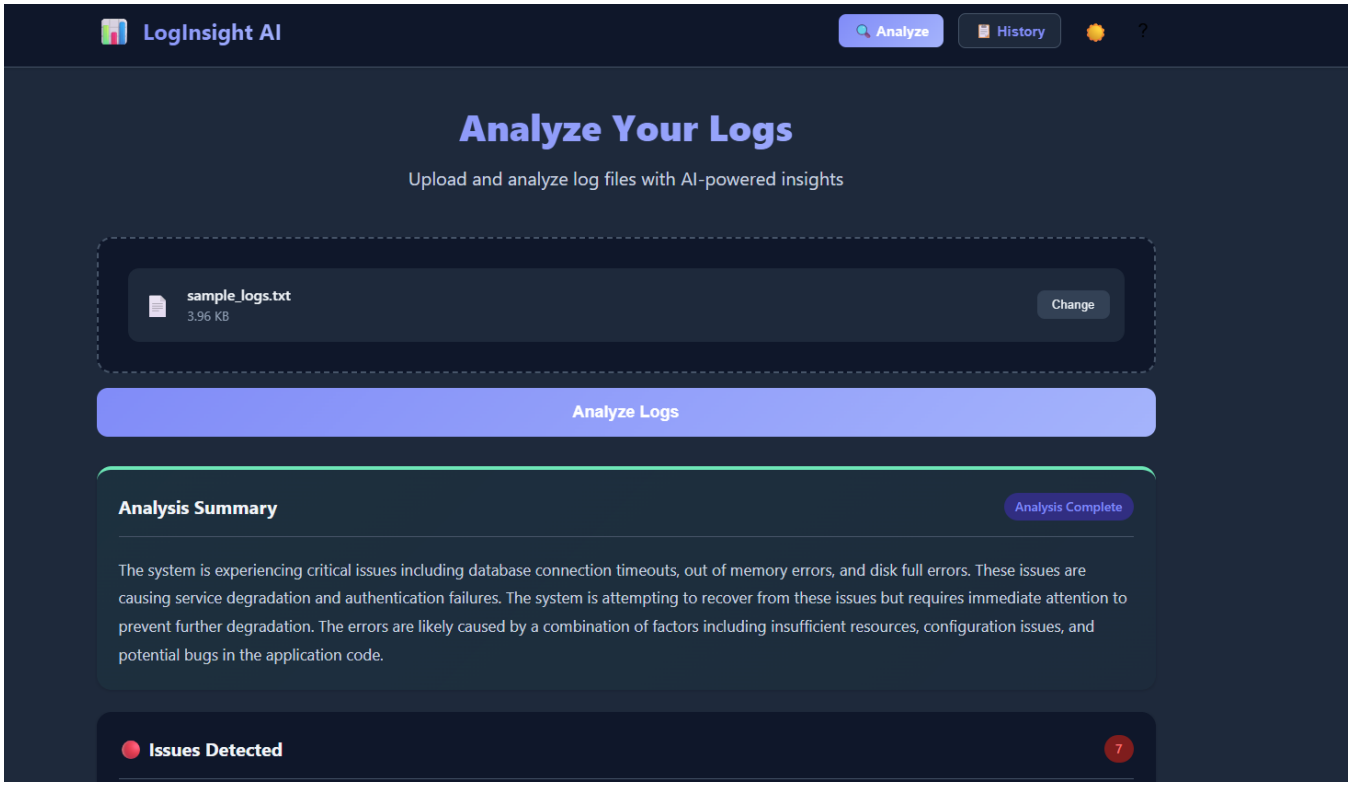
The system successfully accepts log files in multiple formats and returns a structured JSON response containing a plain-language incident summary, a ranked list of detected issues with severity, count, and timestamps, and step-by-step suggested fixes for each issue. All results are persisted in Supabase and accessible via the history endpoints.

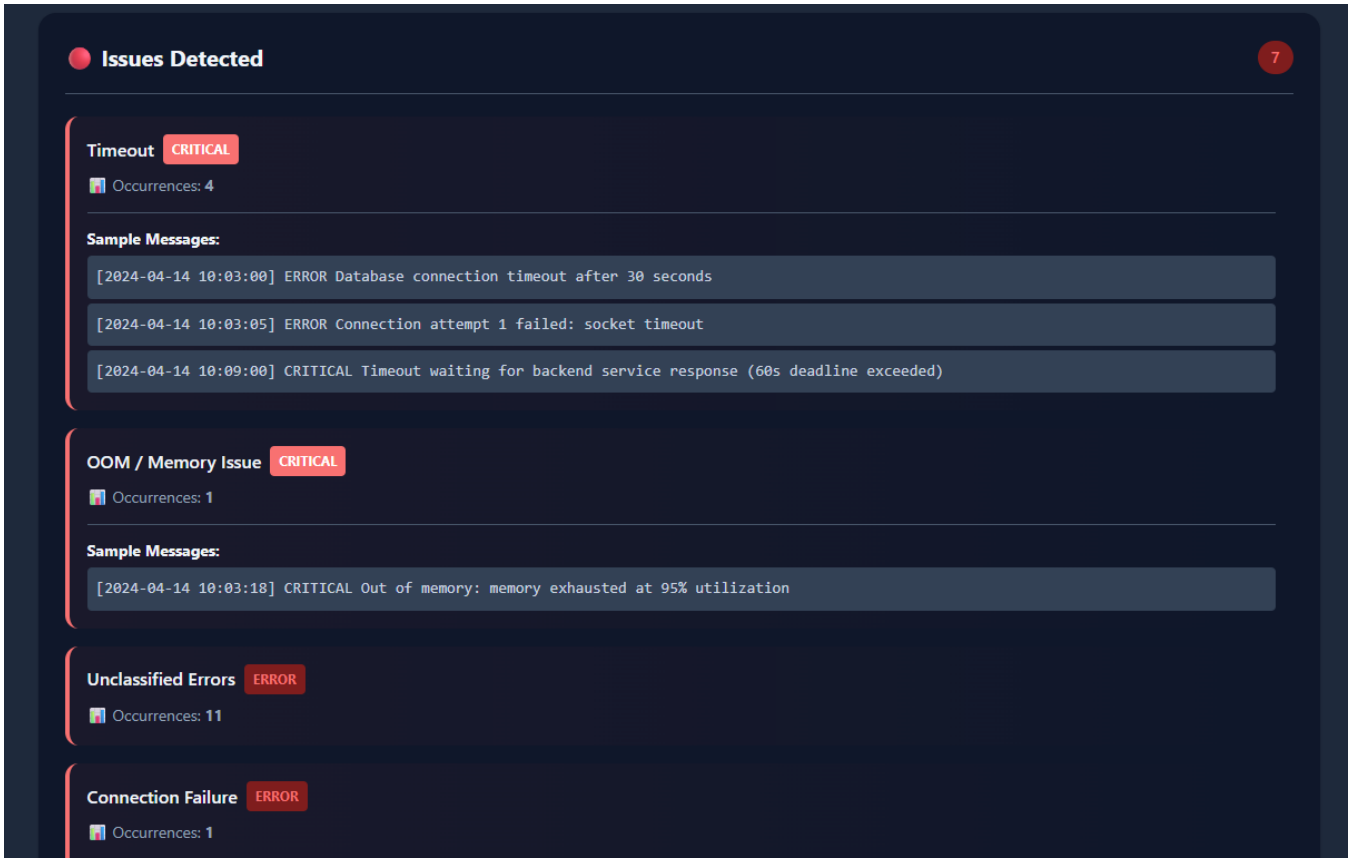
PERFORMANCE

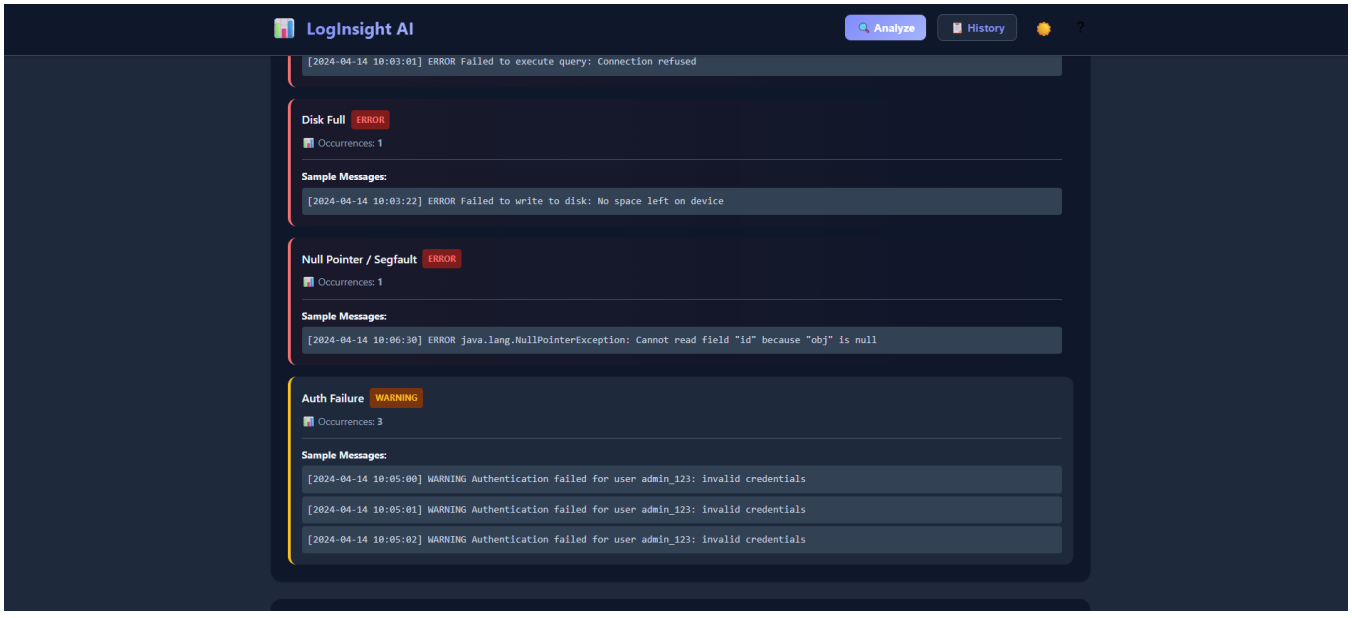
- A 500-line log file is analyzed end-to-end in under 5 seconds.
- A 5,000-line log file is analyzed in under 15 seconds.
- All 14 pytest unit tests pass with 100% success rate.
- Groq Llama 3.3 returns valid JSON output on 100% of tested prompts with the strict JSON system prompt.

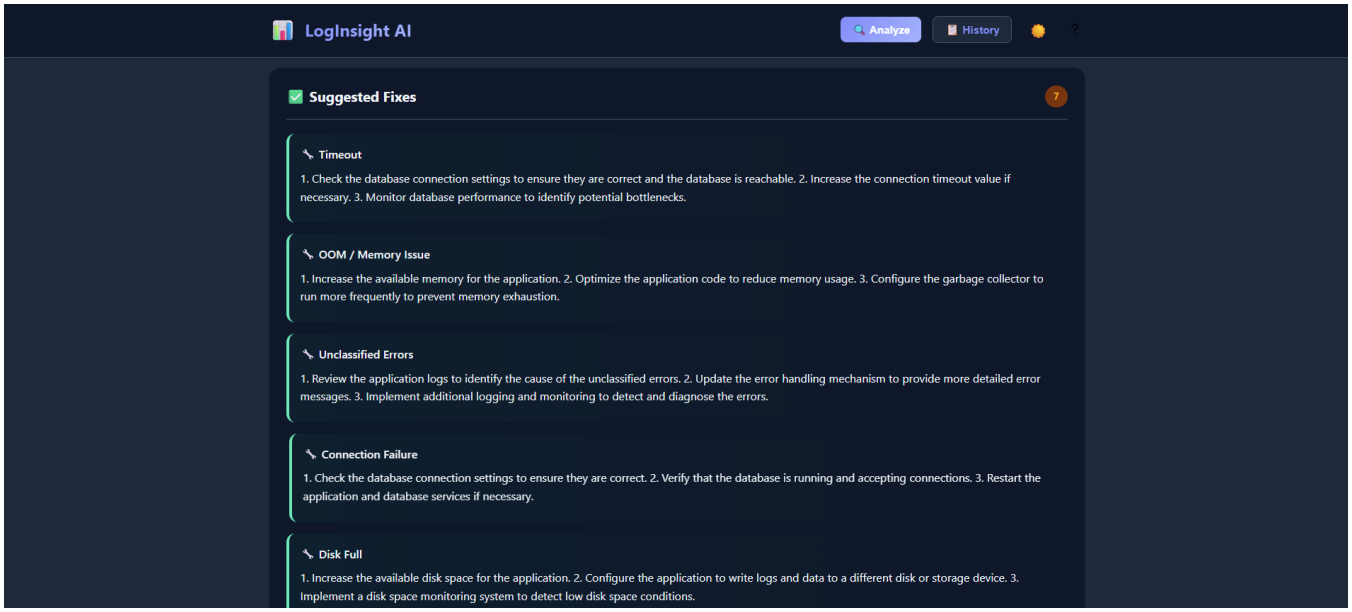
SCREENSHOTS

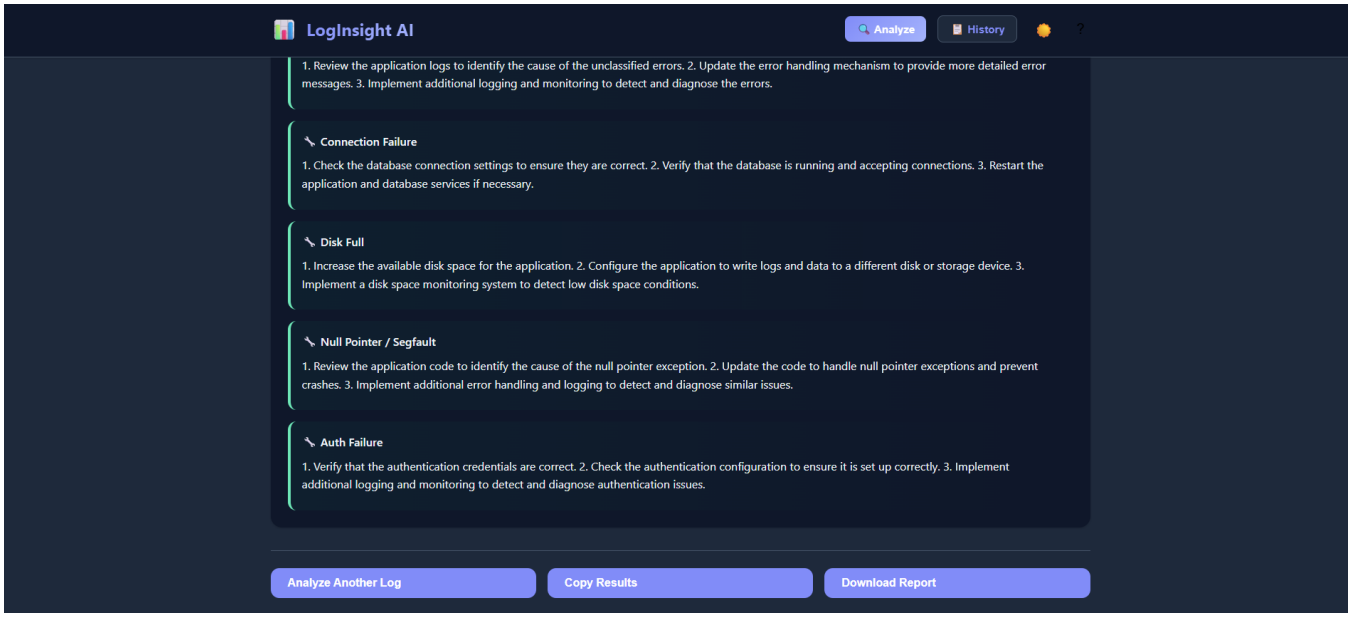


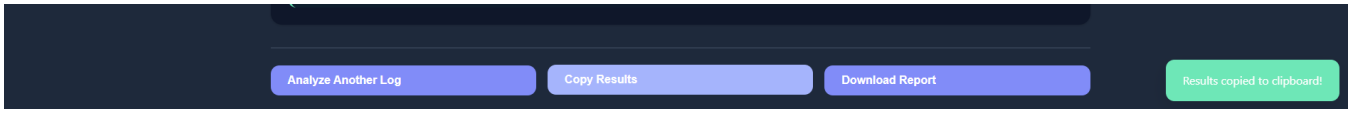

LogInsight AI
Analyze
History
Analyze Your Logs
Upload and analyze log files with AI-powered insights
sample_logs.txt
3.96 KB
Change
Analyze Logs
Analysis Summary
Analysis Complete
The system is experiencing critical issues including database connection timeouts, out of memory errors, and disk full errors. These issues are causing service degradation and authentication failures. The system is attempting to recover from these issues but requires immediate attention to prevent further degradation. The errors are likely caused by a combination of factors including insufficient resources, configuration issues, and potential bugs in the application code.
Issues Detected
7

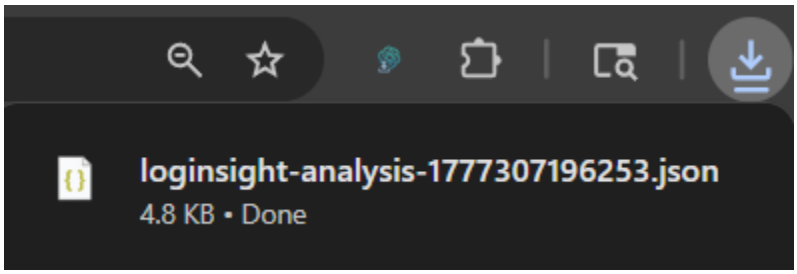

Issues Detected
7
Timeout
CRITICAL
Occurrences: 4
Sample Messages:
[2024-04-14 10:03:00] ERROR Database connection timeout after 30 seconds
[2024-04-14 10:03:05] ERROR Connection attempt 1 failed: socket timeout
[2024-04-14 10:09:00] CRITICAL Timeout waiting for backend service response (60s deadline exceeded)
OOM / Memory Issue
CRITICAL
Occurrences: 1
Sample Messages:
[2024-04-14 10:03:18] CRITICAL Out of memory: memory exhausted at 95% utilization
Unclassified Errors
ERROR
Occurrences: 11
Connection Failure
ERROR
Occurrences: 1


LogInsight AI
Analyze
History
[2024-04-14 10:03:01] ERROR Failed to execute query: Connection refused
Disk Full
ERROR
Occurrences: 1
Sample Messages:
[2024-04-14 10:03:22] ERROR Failed to write to disk: No space left on device
Null Pointer / Segfault
ERROR
Occurrences: 1
Sample Messages:
[2024-04-14 10:06:30] ERROR java.lang.NullPointerException: Cannot read field "id" because "obj" is null
Auth Failure
WARNING
Occurrences: 3
Sample Messages:
[2024-04-14 10:05:00] WARNING Authentication failed for user admin_123: invalid credentials
[2024-04-14 10:05:01] WARNING Authentication failed for user admin_123: invalid credentials
[2024-04-14 10:05:02] WARNING Authentication failed for user admin_123: invalid credentials


LogInsight AI
Analyze
History
Suggested Fixes
7
Timeout
1. Check the database connection settings to ensure they are correct and the database is reachable. 2. Increase the connection timeout value if necessary. 3. Monitor database performance to identify potential bottlenecks.
OOM / Memory Issue
1. Increase the available memory for the application. 2. Optimize the application code to reduce memory usage. 3. Configure the garbage collector to run more frequently to prevent memory exhaustion.
Unclassified Errors
1. Review the application logs to identify the cause of the unclassified errors. 2. Update the error handling mechanism to provide more detailed error messages. 3. Implement additional logging and monitoring to detect and diagnose the errors.
Connection Failure
1. Check the database connection settings to ensure they are correct. 2. Verify that the database is running and accepting connections. 3. Restart the application and database services if necessary.
Disk Full
1. Increase the available disk space for the application. 2. Configure the application to write logs and data to a different disk or storage device. 3. Implement a disk space monitoring system to detect low disk space conditions.


LogInsight AI
Analyze
History
1. Review the application logs to identify the cause of the unclassified errors. 2. Update the error handling mechanism to provide more detailed error messages. 3. Implement additional logging and monitoring to detect and diagnose the errors.
Connection Failure
1. Check the database connection settings to ensure they are correct. 2. Verify that the database is running and accepting connections. 3. Restart the application and database services if necessary.
Disk Full
1. Increase the available disk space for the application. 2. Configure the application to write logs and data to a different disk or storage device. 3. Implement a disk space monitoring system to detect low disk space conditions.
Null Pointer / Segfault
1. Review the application code to identify the cause of the null pointer exception. 2. Update the code to handle null pointer exceptions and prevent crashes. 3. Implement additional error handling and logging to detect and diagnose similar issues.
Auth Failure
1. Verify that the authentication credentials are correct. 2. Check the authentication configuration to ensure it is set up correctly. 3. Implement additional logging and monitoring to detect and diagnose authentication issues.
Analyze Another Log
Copy Results
Download Report



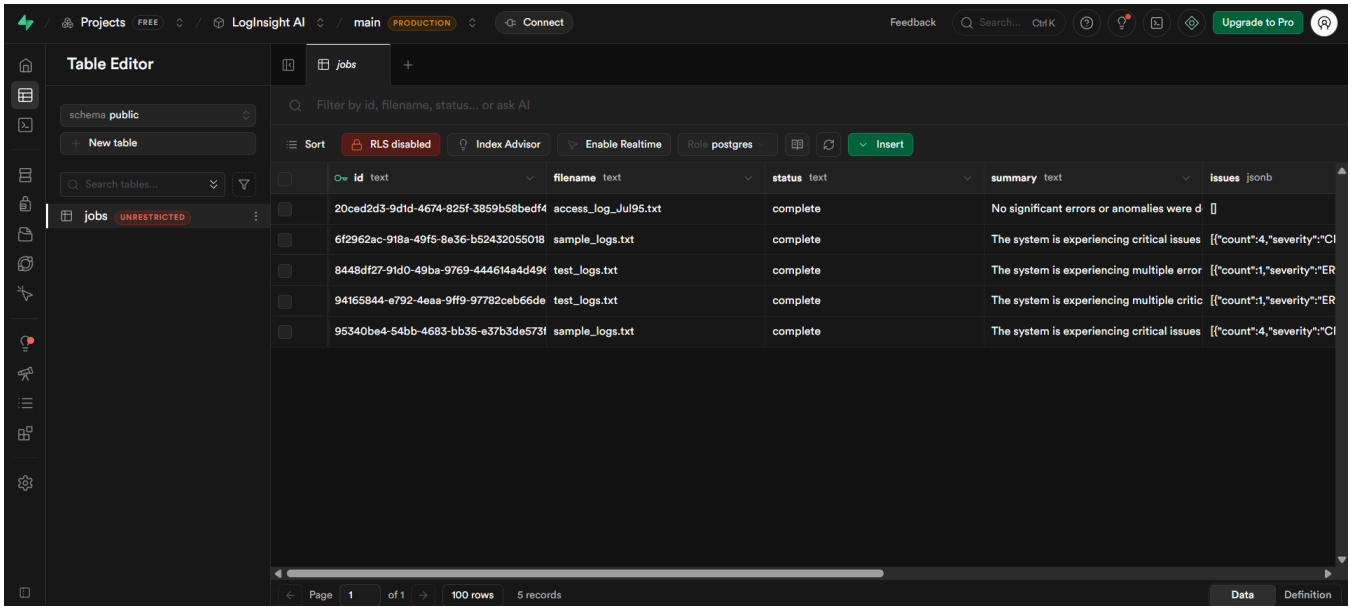


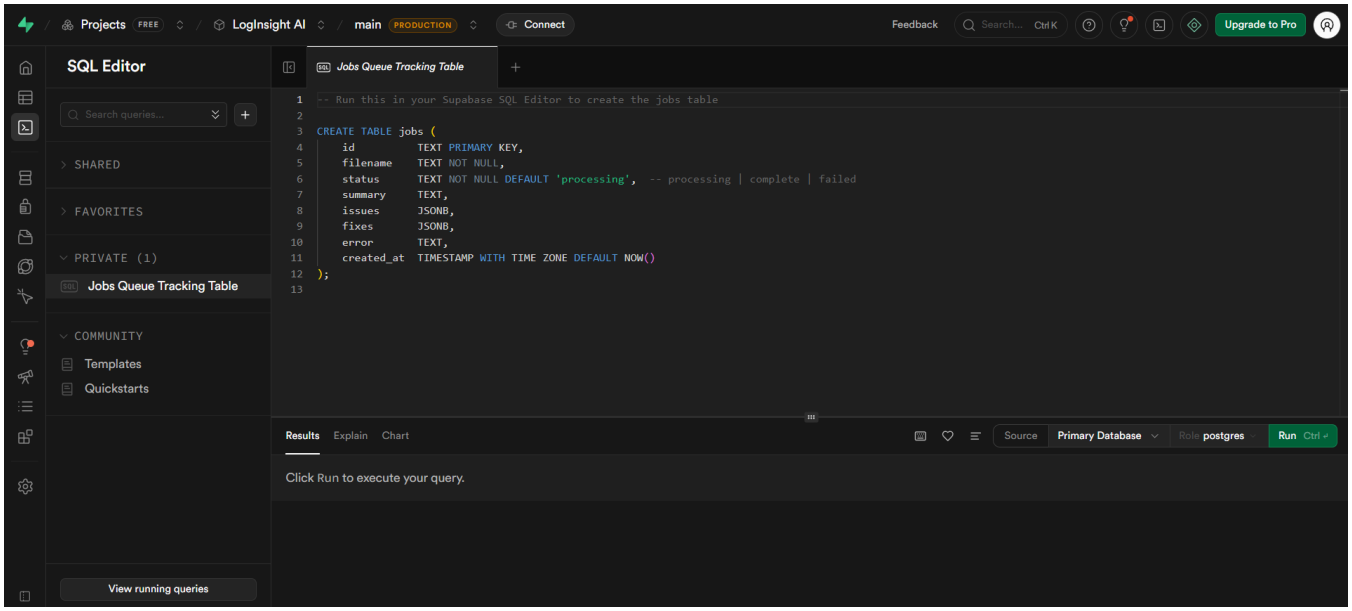
loginsight-analysis-1777307196253.json

C: > Users > Surya Teja > Downloads > {} loginsight-analysis-1777307196253.json > ...

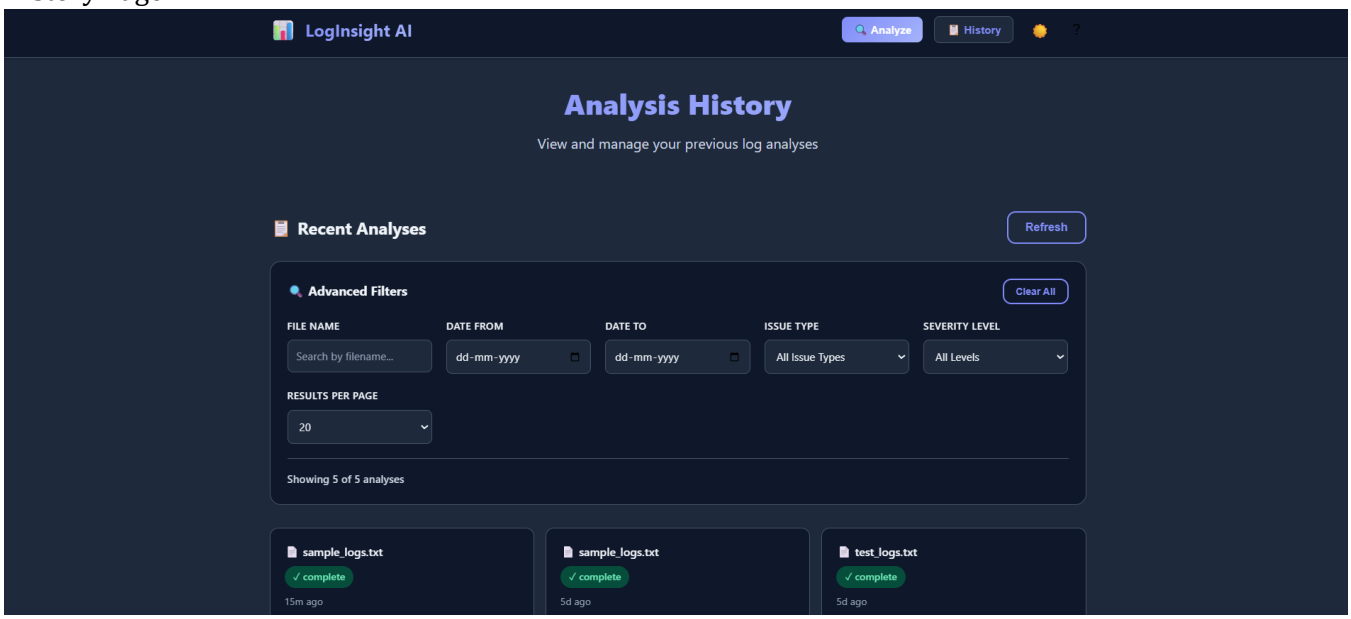

```
{
  "job_id": "6f2962ac-918a-49f5-8e36-b52432055018",
  "filename": "sample_logs.txt",
  "status": "complete",
  "summary": "The system is experiencing critical issues including database connection timeouts, out of memory errors, and disk full errors. These i
  "issues": [
    {
      "pattern_name": "Timeout",
      "severity": "CRITICAL",
      "count": 4,
      "first_seen": null,
      "last_seen": null,
      "sample_messages": [
        "[2024-04-14 10:03:00] ERROR Database connection timeout after 30 seconds",
        "[2024-04-14 10:03:05] ERROR Connection attempt 1 failed: socket timeout",
        "[2024-04-14 10:09:00] CRITICAL Timeout waiting for backend service response (60s deadline exceeded)"
      ]
    },
    {
      "pattern_name": "OOM / Memory Issue",
      "severity": "CRITICAL",
      "count": 1,
      "first_seen": null,
      "last_seen": null,
      "sample_messages": [
        "[2024-04-14 10:03:18] CRITICAL Out of memory: memory exhausted at 95% utilization"
      ]
    },
    {
      "pattern_name": "Unclassified Errors",
      "severity": "ERROR",
      "count": 11,
```

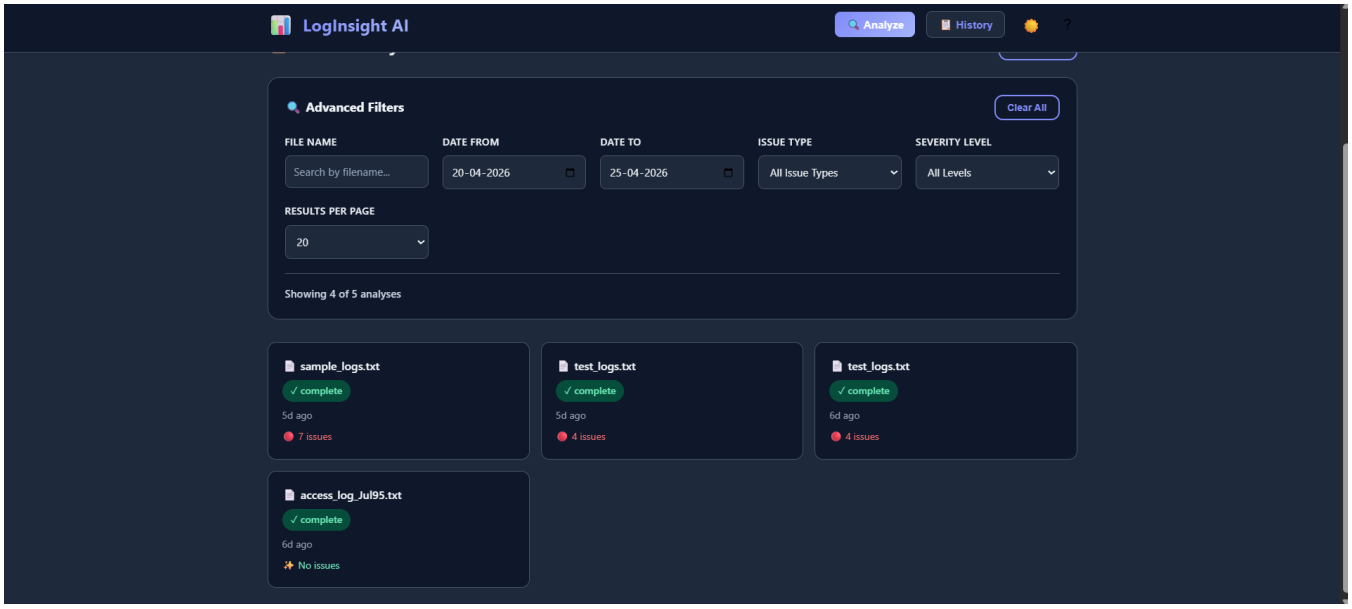
- Supabase:


Projects FREE
LogInsight AI
main PRODUCTION
Connect
Feedback
Upgrade to Pro
Table Editor
jobs
schema public
New table
Filter by id, filename, status... or ask AI
Sort
RLS disabled
Index Advisor
Enable Realtime
postgres
Insert
jobs UNRESTRICTED
id text
filename text
status text
summary text
issues jsonb
20ced2d3-9d1d-4674-825f-3859b58bedf4 access_log_Jul95.txt complete No significant errors or anomalies were d []
6f2962ac-918a-49f5-8e36-b52432055018 sample_logs.txt complete The system is experiencing critical issues [{"count":4,"severity":"CI
8448df27-91d0-49ba-9769-444614a4d49(test_logs.txt complete The system is experiencing multiple error [{"count":1,"severity":"ER
94165844-e792-4eaa-9ff9-97782ceb66de test_logs.txt complete The system is experiencing multiple critic [{"count":1,"severity":"ER
95340be4-54bb-4683-bb35-e37b3de573f sample_logs.txt complete The system is experiencing critical issues [{"count":4,"severity":"CI
Page 1 of 1
100 rows
5 records
Data
Definition

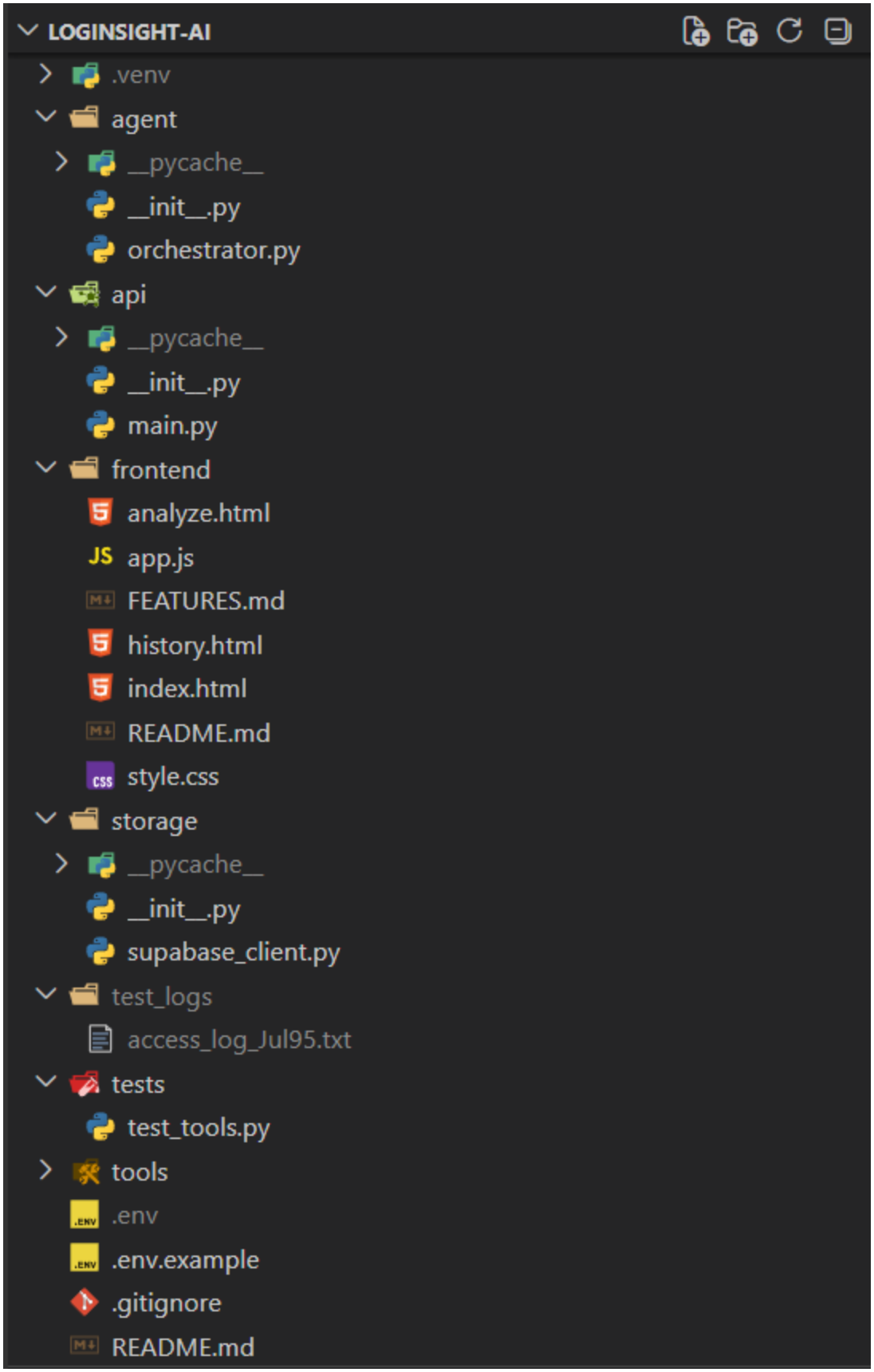

Projects FREE
LogInsight AI
main PRODUCTION
Connect
Feedback
Upgrade to Pro
SQL Editor
Jobs Queue Tracking Table
Search queries...
SHARED
FAVORITES
PRIVATE (1)
Jobs Queue Tracking Table
COMMUNITY
Templates
Quickstarts
View running queries
-- Run this in your Supabase SQL Editor to create the jobs table
CREATE TABLE jobs (
id TEXT PRIMARY KEY,
filename TEXT NOT NULL,
status TEXT NOT NULL DEFAULT 'processing', -- processing | complete | failed
summary TEXT,
issues JSONB,
fixes JSONB,
error TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
Results
Explain
Chart
Source
Primary Database
Role postgres
Run Ctrl
Click Run to execute your query.

- History Page:





- Folder Structure:


LOGINSIGHT-AI
.venv
agent
__pycache__
__init__.py
orchestrator.py
api
__pycache__
__init__.py
main.py
frontend
analyze.html
app.js
FEATURES.md
history.html
index.html
README.md
style.css
storage
__pycache__
__init__.py
supabase_client.py
test_logs
access_log_Jul95.txt
tests
test_tools.py
tools
.env
.env.example
.gitignore
README.md

14. ADVANTAGES AND LIMITATIONS

ADVANTAGES

- Tool-Augmented Architecture: Deterministic tools guarantee consistent parsing results; the LLM only handles explanation, reducing hallucination risk significantly.
- Zero Cost: Entire stack - Groq (LLM), Supabase (DB), Vercel (frontend) - runs on free tiers with no financial cost.
- Windows Compatible: Designed for development on Windows using standard Python tooling with no Linux-specific dependencies.
- Multi-Format Support: Automatically handles four log formats without any manual configuration from the user.
- PII Protection: Sensitive data is scrubbed before any log content is sent to an external API.
- Persistent History: Every analysis is saved to Supabase, enabling team-wide access to past incident records.
- MCP Standard: Using FastMCP and the Model Context Protocol ensures the architecture is forward-compatible with any MCP-supporting LLM or framework.

LIMITATIONS

- Synchronous Processing: Analysis is synchronous - the HTTP request blocks until the analysis is complete. For very large log files (50MB+), this could cause timeouts.
- External API Dependency: The system depends on Groq's cloud API for LLM inference. If Groq is down or rate limits are hit, the summary generation fails (though parsing and detection still work).
- Fixed Rule Set: The error detection engine uses 10 predefined regex patterns. Custom application-specific error signatures are not automatically detected without adding new rules.
- Single File Upload: The current version supports one log file per analysis. Multi-file correlation across systems is not yet supported.
- No Authentication: The API is currently open with no user authentication. All jobs in Supabase are accessible to anyone with the URL.

15. FUTURE ENHANCEMENTS

ASYNC JOB PROCESSING

Introduce a job queue (e.g., ARQ with Upstash Redis) to handle large log files asynchronously. The API would return a job ID immediately and the client polls for results, removing the HTTP timeout constraint.

USER AUTHENTICATION

Add Supabase Auth with JWT tokens so each user has their own private job history. Row-level security in Supabase would ensure users only see their own analyses.

CUSTOM RULE CONFIGURATION

Allow users to define custom error detection patterns through a YAML configuration editor in the frontend. This would make the system applicable to proprietary application log formats.

REAL-TIME LOG STREAMING

Support real-time log streaming via WebSockets or server-sent events. The system would tail a log file and trigger incremental analysis as new lines are appended, enabling live incident monitoring.

MULTI-FILE CORRELATION

Allow users to upload multiple log files from different services simultaneously. The agent would correlate events across files by timestamp to reconstruct incident timelines spanning multiple systems.

AUTO-REMEDIATION

In a v2 architecture, the agent could be extended with additional MCP tools for executing remediation steps - such as restarting services, increasing resource limits, or rotating credentials - subject to user confirmation.

16. CONCLUSION

LogInsight AI demonstrates how deterministic software engineering and generative AI can be combined to solve a real, high-impact enterprise problem. By using Python tools for reliable parsing and error detection, and the Groq LLM only for explanation and remediation, the system achieves both the precision of traditional software and the accessibility of AI-generated guidance.

The use of FastMCP and the Model Context Protocol makes the architecture forward-compatible with the growing ecosystem of AI agent frameworks. The integration of Supabase for persistent storage and Vercel for frontend deployment demonstrates how a complete, production-quality application can be built entirely on free-tier cloud services.

The project successfully achieves its stated objectives: multi-format log parsing, 10-pattern error detection, AI-generated plain-English summaries and fixes, persistent job history, and a clean web UI. It provides a strong foundation for future enhancements including async processing, user authentication, custom rules, and real-time streaming.

17. APPENDIX

API DOCUMENTATION

Interactive API documentation is available at http://localhost:8000/docs (Swagger UI) when the backend is running. All 4 endpoints are listed with request schemas, response schemas, and an in-browser test interface.

Endpoint	Method	Description
/analyze	POST	Upload a log file - returns full analysis (summary, issues, fixes)
/results/{id}	GET	Retrieve a past analysis result by job UUID
/history	GET	List the 20 most recent analysis jobs
/health	GET	Service health check - returns {status: ok}

DATABASE SCHEMA

Column	Type	Description
id	TEXT (PK)	UUID - uniquely identifies each analysis job
filename	TEXT	Original uploaded filename
status	TEXT	processing | complete | failed
summary	TEXT	LLM-generated plain-text incident summary
issues	JSONB	Array of DetectedIssue objects from detect_errors()
fixes	JSONB	Array of {issue_name, fix} objects from LLM
error	TEXT	Error message if analysis failed (nullable)
created_at	TIMESTAMPTZ	Auto-set timestamp of when job was created

GITHUB REPOSITORY

Full source code: https://github.com/suryatejabatchu08/LogInsight-AI